

Practical Assignment - Staff Engineer - Mobile Apps (Architect - Mobile Apps)

Overview

Your task is to build **PulseCrypto**, a real-time cryptocurrency market viewer consisting of:

- A **Node.js backend** that ingests and processes live market data.
- A **React Native mobile application** that visualizes the processed data in real time.

The objective of this exercise is **not** to produce the most visually polished application. Instead, we are evaluating your ability to design and build a responsive, maintainable system that performs well under sustained real-time updates.

You are encouraged to use AI-assisted development tools (Cursor, Claude Code, GitHub Copilot, ChatGPT, etc.). We will evaluate the final solution, your architectural decisions, and how effectively you leverage AI—not whether every line of code was written manually.

The UI mockup for this application can be found here:



Technology Requirements

Backend

- Node.js
- TypeScript preferred
- Express, Fastify, or similar
- WebSockets

Mobile

- React Native
 - Expo or React Native CLI
 - Must run on:
 - Android Emulator (required)
 - iOS Simulator (optional)
-

Functional Requirements

Part 1 – Backend

Build a Node.js service that acts as a market data gateway between the Binance public market feed and the mobile application.

1. Live Market Streams

Subscribe to the Binance public market streams for **multiple trading pairs**.

At a minimum, support:

- BTC/USDT
- ETH/USDT
- SOL/USDT
- DOGE/USDT
- XRP/USDT

You may support additional pairs if desired.

2. Stream Processing

Raw market updates can arrive many times per second.

Your backend should:

- Connect to the Binance WebSocket market streams
- Continuously ingest order book updates
- Buffer and/or batch incoming updates
- Emit processed updates to connected mobile clients at a configurable interval (default: **100ms**)
- Prevent slow consumers from causing unbounded memory growth

The buffering strategy is your choice and should be explained in your README.

3. WebSocket Server

Expose a local WebSocket server that broadcasts processed market updates.

Each update should identify which trading pair it belongs to.

Example payload:

```
1 {  
2   "pair": "BTCUSDT",  
3   "timestamp": 1720802025,
```

```
4   "price": 109235.42,  
5   "spread": 0.41,  
6   "buyPressure": 63,  
7   "sellPressure": 37,  
8   "bids": [...],  
9   "asks": [...]  
10 }
```

The exact payload format is up to you but should be documented.

4. REST Endpoint

Expose:

```
1 GET /pairs/meta
```

Return metadata for all supported trading pairs.

Example:

- Display Name
- Trading Status
- 24 Hour High
- 24 Hour Low
- 24 Hour Volume

This data may be mocked.

Part 2 – Mobile Application

Build a React Native application that consumes the backend.

1. Market Watchlist

Display all supported trading pairs.

Each row should display:

- Trading Pair
- Current Price
- 24 Hour Change
- Live Connection Indicator
- Favourite Toggle

Example:

```
1 BTC / USDT ▲ 1.82%
```

```
2
3 ETH / USDT      ▼ 0.41%
4
5 SOL / USDT      ▲ 4.12%
6
7 DOGE / USDT     ▲ 0.83%
8
9 XRP / USDT      ▼ 1.22%
```

2. Search

Allow users to search or filter trading pairs.

Example:

```
1 Search...
2
3 btc
4
5 ↓
6
7 BTC / USDT
```

3. Favourites

Allow users to favourite trading pairs.

Requirements:

- Persist favourites locally
 - Restore favourites when the application restarts
-

4. Market Details

Selecting a trading pair should display:

- Current Price
 - Buy Pressure
 - Sell Pressure
 - Spread
 - Live Order Book (Bids & Asks)
 - Last Updated Timestamp
-

5. Live Updates

The application should receive continuous updates from the backend.

The interface should remain smooth and responsive even during sustained update bursts.

When data changes:

- Price increases should briefly highlight in green
- Price decreases should briefly highlight in red
- Order book volume changes should animate smoothly

The implementation details are left to you.

6. Offline Behaviour

If the backend becomes unavailable:

- Display the current connection status
 - Continue showing the most recently received data
 - Automatically reconnect when connectivity is restored
-

7. Pull to Refresh

Support pull-to-refresh to reload metadata from:

```
1 GET /pairs/meta
```

without interrupting the live WebSocket stream.

Non-Functional Requirements

Your solution should demonstrate:

- Clean architecture
- Maintainable code
- Responsive UI under continuous updates
- Efficient state management
- Robust connection handling
- Appropriate error handling
- Good separation of concerns

You are free to choose the libraries and architectural patterns you believe are appropriate.

Deliverables

Please provide:

- Complete source code shared via a Git repo
- Screen recording of the app.
- README containing:
 - Setup instructions
 - Build and run instructions
 - Architectural decisions
 - Assumptions made
 - Trade-offs considered
 - How AI-assisted development tools were used